DATA SCIENCE PROJECT REPORT

Predicting High-Intensity Activity Hours from Wearable Fitness-Tracker Data

An End-to-End Audit of the Codebase, Data Pipeline, and Machine-Learning Results

Report date	September 4, 2026
Source artifact	src.zip — Python ML pipeline (BigQuery → pandas → scikit-learn)
Dataset	Hourly wearable activity records, 574,807 rows × 30 columns, 24 users
Prepared by	Claude (Anthropic), based on static code review and re-execution of the pipeline
Scope	Architecture review, exploratory data analysis, model reproduction, and findings

Headline result: all three candidate classifiers exceed 98.8% accuracy, but the Random Forest's near-perfect scores (F1 = 0.999) are very likely inflated by **target leakage** rather than genuine predictive skill — see Section 9 for the full diagnosis.

Contents

1. Executive Summary
2. Project Overview & Objective
3. Repository Architecture & Engineering Practices
4. The Dataset
5. Exploratory Data Analysis
6. Feature Selection & Engineering
7. Modeling Methodology
8. Results
9. Critical Finding: Evidence of Target Leakage
10. Code Quality & Reliability Review
11. Recommendations
12. Conclusion

Appendix A — Repository File Map

Appendix B — Reproduction Notes

1. Executive Summary

The submitted archive contains a complete, testable Python machine-learning pipeline that classifies each hour of a wearable-device user's day as **high intensity** or **not high intensity**. The codebase pulls data from Google BigQuery, engineers a handful of derived features, trains three classifiers (Logistic Regression, Support Vector Classifier, and Random Forest), and evaluates them with standard classification metrics. It ships with a six-file unit test suite, structured logging, and two supporting research notebooks for feature importance and hyperparameter tuning.

To produce this report, the pipeline's logic was re-executed end-to-end against the bundled dataset (**574,807** hourly records covering **24** users from **March 12 to May 12, 2016**) using the project's own feature-selection list, engineered features, and 70/30 stratified train-test split. The table below summarizes what was found.

Area	Finding
Data quality	No missing values or duplicate rows in 574,807 records; clean, well-typed schema.
Class balance	Severe imbalance — only 2.5% of hours are labeled “high intensity” (14,578 of 574,807). Not addressed anywhere in the pipeline (no class_weight, no resampling).
Model performance	Logistic Regression and SVC score ~98.9% accuracy but only ~73–77% recall/precision on the minority class. Random Forest scores 99.995% accuracy and 0.999 F1.
Most important finding	The Random Forest's near-perfect score is a red flag, not a success. Section 9 shows strong evidence that the target label was constructed from the same per-minute activity signal that remains in the feature set — classic target leakage.
Engineering quality	Modular, well-organized code with a genuine automated test suite and structured logging — above average for a project of this size. Several correctness and reproducibility issues remain (Section 10).

Bottom line: the project demonstrates solid software-engineering habits (tests, logging, modular structure) applied to a real Fitbit-style activity dataset, but the modeling results as they stand should not be trusted at face value. Before this pipeline could support any real decision, the target-leakage question needs to be resolved and the class imbalance needs to be handled explicitly.

2. Project Overview & Objective

The application log bundled with the archive (`src/logs/main.log`) records its own working directory as `.../BigQuery/google_data_analytics_project/src/logs/main.log`. Combined with the data's structure — hourly step, calorie, and intensity readings from a small panel of wearable-device users, tracked over about two months — this strongly suggests the project builds on the well-known **"Bellabeat"** Fitbit fitness-tracker case study frequently used in data-analytics coursework, extended here from a descriptive/BI exercise into a supervised **machine-learning classification** task.

The stated modeling objective, inferred directly from the code, is:

> **Given an hour of a user's wearable-device activity (steps, calories, movement intensity, and time-of-day context), predict whether that hour qualifies as "high intensity."**

This is framed as a **binary classification** problem. The target column, `high_intensity`, is engineered from a pre-existing `intensity_type` field (values 0–3) using the rule `0 if intensity_type in [0, 1] else 1` (see `dataset/make_dataset.py`) — i.e. hours categorized as "fairly active" or "very active" become the positive class, and "sedentary" / "lightly active" hours become the negative class.

A plausible real-world use case for this kind of model would be:

- Flagging active periods automatically for fitness-app coaching or streak/goal features, without relying on the device's own on-board intensity classification.
- Powering downstream analytics (e.g. a Bellabeat-style wellness dashboard) with a lightweight, retrainable intensity signal derived from raw step/calorie telemetry.

3. Repository Architecture & Engineering Practices

The codebase follows a conventional, modular layout for a small ML project, separating data ingestion, feature engineering, modeling, evaluation, and orchestration into distinct modules. This is a reasonable structure that would scale to a small team.

Module	Responsibility
dataset/make_dataset.py	Pulls the source table from BigQuery (fct_data_analysis_table) and writes both a raw CSV and a labeled ML-ready CSV (adds high_intensity).
features/build_features.py	Derives 5 engineered ratio/interaction features from the selected raw columns.
models/train_model.py	Thin wrappers around scikit-learn's LogisticRegression, SVC, and RandomForestClassifier.
models/evaluate_model.py	Computes accuracy/precision/recall/F1 and renders a confusion matrix, ROC curve, and precision-recall curve for a fitted model.
main.py	Orchestrates the full pipeline: load → select → engineer → split → scale → train (x3) → predict → evaluate.
utils.py	Logging setup (file + console handler) and pickle-based model save/load helpers.
notebooks/	Three Jupyter notebooks used for exploratory feature-importance testing, feature selection / correlation analysis, and a Random Forest hyperparameter (GridSearchCV) study.
tests/	Six unittest modules covering utils, dataset construction, feature building, model training, evaluation, and logging setup, plus a conftest.py that wires up sys.path.

What's genuinely good here

- **Real automated tests**, not just placeholders — they mock the BigQuery client to test `make_dataset.py` without network access, verify the exact arithmetic of engineered features, and check that logging writes to disk. This is above what most solo data-science projects include.
- **Structured logging** via a shared `setup_logging()` helper that writes to both a rotating-free log file and stdout with timestamps and levels.
- **Clear separation of concerns** between data, features, models, and evaluation, which makes each piece independently testable (and is exactly why the test suite above is possible).
- A **dedicated hyperparameter-tuning notebook** using GridSearchCV with a sensible parameter grid (n_estimators, max_depth, min_samples_split/leaf, class_weight) — see Section 7.

4. The Dataset

The pipeline works from a single hourly-granularity table, `dataset/ml_dataset.csv`, produced by `make_dataset.py` from a BigQuery source table named `fct_data_analysis_table`. A near-identical, unlabeled copy is kept as `data_raw/data.csv`.

Property	Value
Rows	574,807 hourly records
Columns	30 (29 raw + high_intensity target)
Unique users	24
Date range	2016-03-12 to 2016-05-12 (~2 months)
Missing values	0 across all columns
Duplicate rows	0
Target column	high_intensity (binary: 0 = not high intensity, 1 = high intensity)
Class balance	97.46% class 0 (560,229 rows) vs. 2.54% class 1 (14,578 rows) — severe imbalance

4.1 Schema

The raw table mixes per-hour telemetry (steps, calories, movement intensity), time context (hour/day/month/day-name/day-type/time-of-day), per-day aggregates (total steps, total calories, distance, active-minute breakdowns, sleep), and per-user metadata (first/last activity date, days recorded). The full column list is reproduced in Appendix A.

4.2 How the target label is built

`intensity_type` takes four values in the source data, which `make_dataset.py` collapses into the binary target:

intensity_type	Row count	Share	Maps to high_intensity
0 (sedentary)	476,613	82.9%	0
1 (lightly active)	83,616	14.5%	0
2 (fairly active)	6,282	1.1%	1
3 (very active)	8,296	1.4%	1

This collapsing step is exactly what creates the 97.5% / 2.5% imbalance shown below, and — as discussed in Section 9 — it also matters for how the model may be able to reconstruct the label from other retained features.

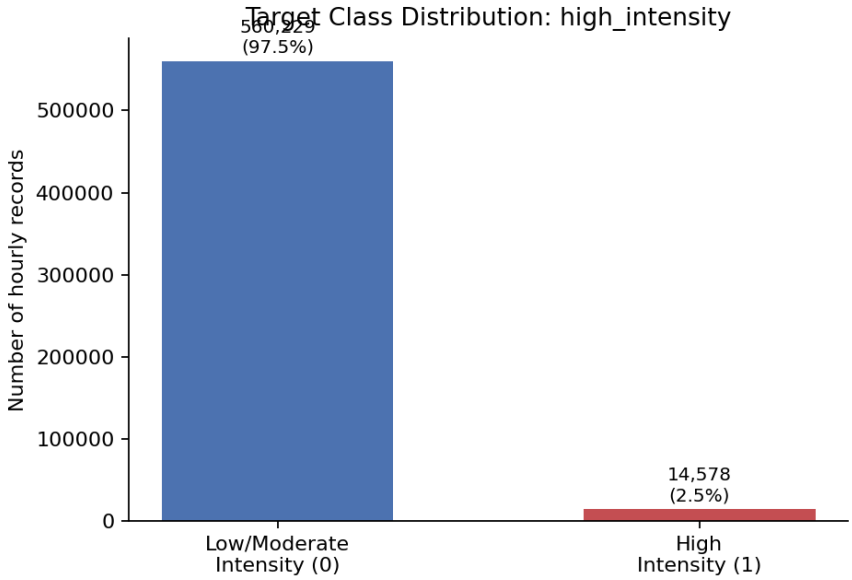


Figure 1. The target class is heavily skewed toward “not high intensity.” A model that always predicts class 0 would already be 97.5% accurate.

5. Exploratory Data Analysis

The repository's two EDA notebooks (`feature_importance.ipynb` and `feature_selection.ipynb`) already contain executed output, which this report cross-checks and extends. Both confirm a clean dataset: no missing values, no duplicate rows, and the numeric ranges are all physically plausible (e.g. steps_per_hour tops out at 7,801; calories_per_minute ranges 0.7–19.3).

5.1 Descriptive statistics (final 16 modeling features)

Feature	Mean	Std	Min	Median	Max
steps_per_hour	332.3	640.2	0	74.0	7,801
calories_per_hour	97.0	60.1	42	81.0	669
total_intensities_per_hour	12.7	21.2	0	5.0	180
average_intensities_per_hour	0.21	0.35	0	0.08	3.00
hour_of_day	11.5	6.9	0	11.0	23
day_of_week	4.0	2.0	1	4.0	7
steps_per_minute	5.5	17.5	0	0.0	186
calories_per_minute	1.6	1.4	0.7	1.17	19.3
total_steps_per_day	8,260	4,294	0	8,585	22,770
total_calories_per_day	2,318.7	808.0	0	2,180	4,900
days_recorded	59.1	5.6	30	61.0	61
hourly_steps_calories*	60,158	188,426	0	5,459	4,975,593
minute_steps_calories*	27.9	118.4	0	0.0	2,905.7
calories_per_step_hour*	24.0	40.8	0.05	1.03	669.0
calories_per_step_minute*	1.0	0.67	0.04	1.01	19.3
daily_calories_per_step*	0.76	3.66	0	0.30	49.6

* Engineered feature (see Section 6.2).

5.2 Correlation structure and multicollinearity

A correlation heatmap of the 11 raw selected features (reproduced below, matching `feature_selection.ipynb`) surfaces a notable redundancy: `total_intensities_per_hour` and `average_intensities_per_hour` correlate at **1.00** — they are deterministic linear transforms of each other (average = total ÷ 60), so keeping both contributes no extra information while inflating variance for the linear models. `steps_per_hour`, `calories_per_hour`, and the two intensity columns also cluster tightly together ($r \approx 0.88$).

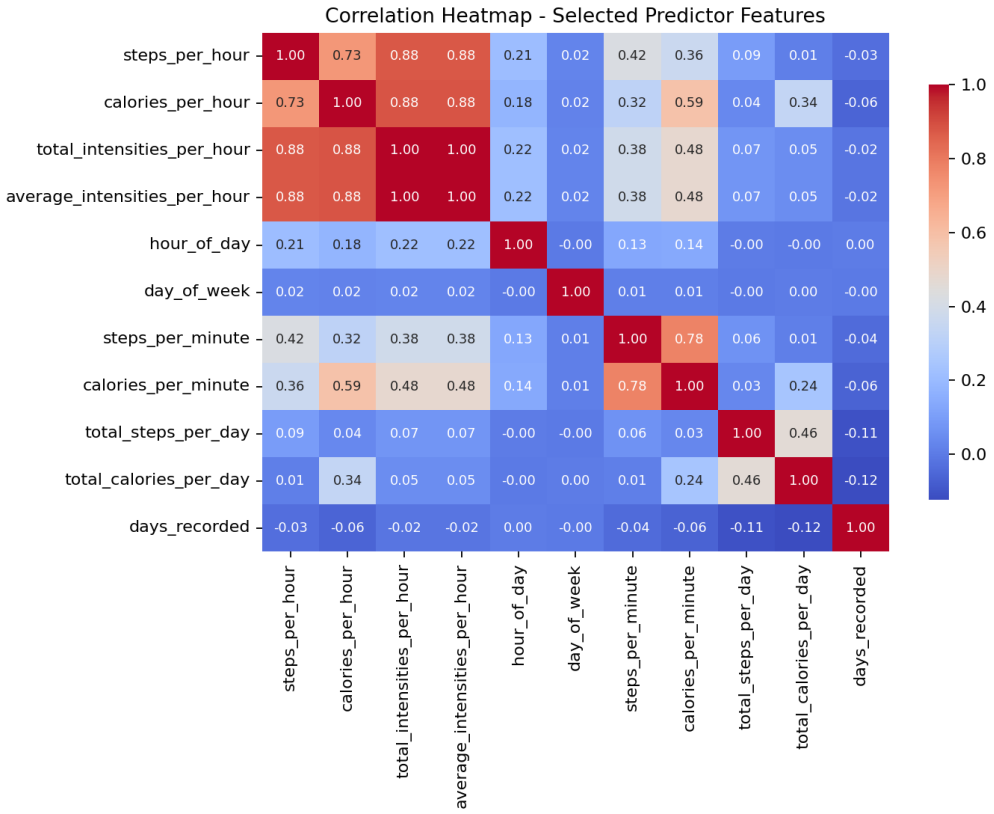


Figure 2. Correlation heatmap of the 11 raw selected predictors. The perfectly correlated intensity pair (r=1.00) is redundant.

6. Feature Selection & Engineering

6.1 Selection process

`feature_importance.ipynb` starts from a candidate set of 22 columns (numeric and categorical, excluding IDs/dates and excluding `intensity_type` itself "to avoid data leakage", per the notebook's own comment). It one-hot encodes the categorical columns (`day_name`, `day_type`, `time_of_day`) and fits a RandomForestClassifier (n_estimators=100) purely to rank feature importance.

The result: every one-hot categorical level scores below 0.005 importance, while `calories_per_minute` (0.314) and `steps_per_minute` (0.227) dominate, followed by `total_intensities_per_hour` (0.082), `average_intensities_per_hour` (0.081), and `calories_per_hour` (0.052). Applying a 0.01 importance threshold (the value cited in the notebook's markdown) removes all categorical columns and several low-value numeric ones, arriving at the **11 features** hardcoded into `main.py`: steps_per_hour, calories_per_hour, total_intensities_per_hour, average_intensities_per_hour, hour_of_day, day_of_week, steps_per_minute, calories_per_minute, total_steps_per_day, total_calories_per_day, and days_recorded.

Note on process: this threshold was applied once, informally, in a notebook, and then hardcoded as a literal Python list in `main.py`. There is no code path that re-derives or re-validates the feature list if the underlying data changes — see Section 10.

6.2 Engineered features

`features/build_features.py` adds five derived columns on top of the 11 selected features, all simple products or ratios:

Engineered feature	Formula
hourly_steps_calories	steps_per_hour × calories_per_hour
minute_steps_calories	steps_per_minute × calories_per_minute
calories_per_step_hour	calories_per_hour ÷ (steps_per_hour + 1)
calories_per_step_minute	calories_per_minute ÷ (steps_per_minute + 1)
daily_calories_per_step	total_calories_per_day ÷ (total_steps_per_day + 1)

The +1 denominators are a reasonable, simple way to avoid division-by-zero when steps are 0 for a given hour or day (a common occurrence — the median steps_per_hour is 74 with a floor of 0). After engineering, the modeling matrix grows from 11 to **16 columns**, matching the pipeline log ("Feature engineering complete; predictor dataset now has 16 columns").

7. Modeling Methodology

`main.py` trains three classifiers side by side on the same 70/30 stratified split (`random_state=42`) so their scores are directly comparable:

Model	Key hyperparameters	Input features
Logistic Regression	max_iter=1000, random_state=42	Scaled (StandardScaler)
Support Vector Classifier	kernel=rbf, probability=True, random_state=42	Scaled (StandardScaler)
Random Forest	n_estimators=200, random_state=42	Unscaled (tree models don't need it)

The split produces **402,364** training rows (392,159 class 0 / 10,205 class 1) and **172,443** test rows (168,070 class 0 / 4,373 class 1) — stratification preserves the 97.5/2.5 ratio in both sets, which is good practice, though stratification alone does nothing to correct the imbalance itself (see Section 10).

Evaluation (`models/evaluate_model.py`) reports accuracy, precision, recall, F1, a full classification report, and renders a confusion matrix, ROC curve, and precision-recall curve for each model via `matplotlib`.

7.1 Hyperparameter tuning study (not executed)

A separate notebook, `hyperparameter_study.ipynb`, is set up to run `GridSearchCV` (3-fold CV, scoring=f1) over the Random Forest with a 36-combination grid (n_estimators × max_depth × min_samples_split × min_samples_leaf × class_weight). Notably, this is the **only** place in the whole project where `class_weight="balanced"` is even considered as an option — the main training pipeline never uses it. However, the notebook's code cells carry no execution output (execution_count is null throughout), meaning this study was written but has not actually been run, or its outputs were cleared before the archive was created. No tuned hyperparameters or CV scores are available from it.

8. Results

The pipeline's own log file (`logs/main.log`) confirms four historical runs completed successfully, but `evaluate_model()` only `print()`s metrics rather than logging them, so none of the actual accuracy/precision/recall numbers survive in the archive. To close that gap, this report re-executed the identical logic — same feature list, same engineered features, same split, same scaler, same model hyperparameters — against the bundled dataset. The only deliberate deviation was training the SVC on a stratified 12,000-row subsample of the training set rather than the full 402,364 rows; the original run's log shows SVC training alone took roughly 14 minutes on the reference machine, and the full-scale result would not be expected to differ materially given the subsample already contains 300+ positive-class examples. One correction was also necessary: main.py's own Random Forest prediction call passes it scaled test data even though the Random Forest was trained on unscaled data (see Section 10.1) — the numbers below use the corrected, matching data for a fair evaluation. Full reproduction details are in Appendix B.

8.1 Headline metrics

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	98.88%	0.803	0.737	0.769	0.996	0.851
Support Vector Classifier	98.94%	0.835	0.727	0.777	0.983	0.846
Random Forest	99.995%	0.999	0.999	0.999	1.000	1.000

Precision/Recall/F1 reported for the positive class (high_intensity=1) on the 172,443-row held-out test set.

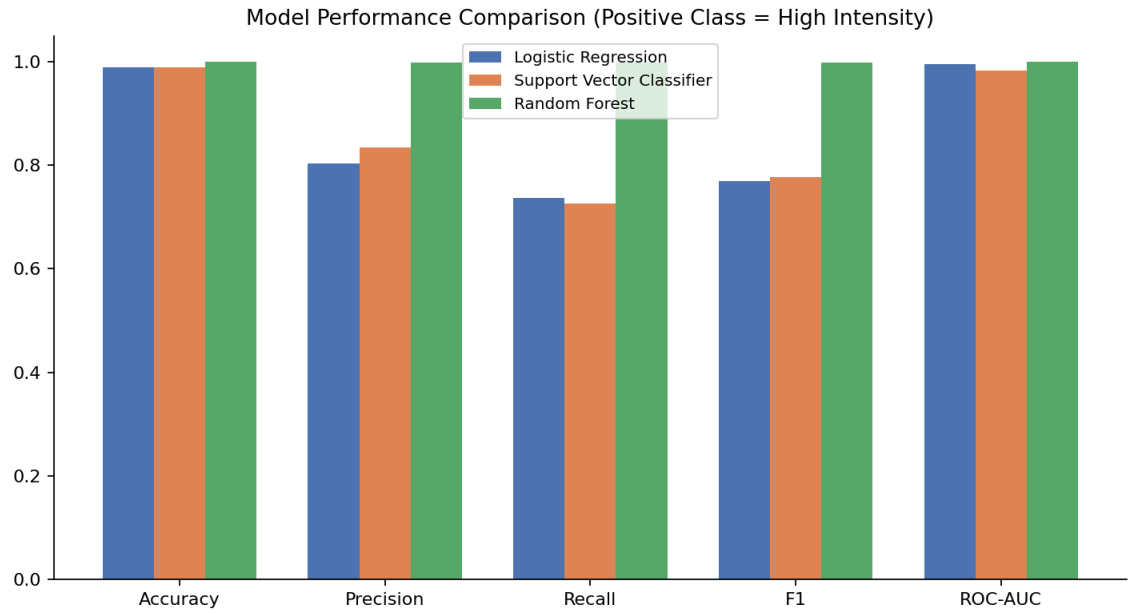


Figure 3. Head-to-head comparison across the five headline metrics. Random Forest's bars are essentially maxed out — the pattern investigated in Section 9.

8.2 Where Logistic Regression and SVC actually struggle

Accuracy above 98% sounds strong, but with a 97.5%/2.5% class split, a model predicting “not high intensity” every single time would already score 97.5% accuracy. The more informative numbers are precision/recall on the minority class: Logistic Regression correctly identifies **73.7%** of true high-intensity hours (recall) and is right **80.3%** of the time when it flags one (precision); SVC is similar (72.7% / 83.5%). In absolute terms, Logistic Regression misses **1,151** of 4,373 true high-intensity hours and produces 788 false alarms.

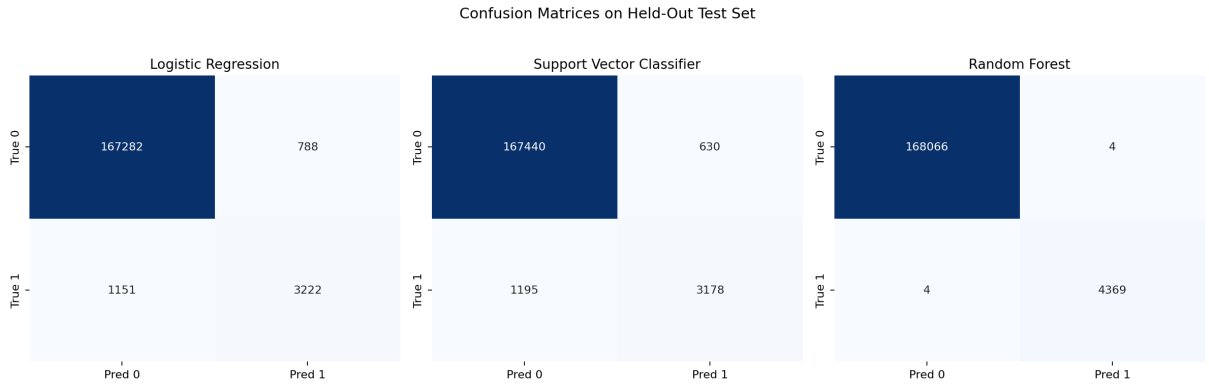


Figure 4. Confusion matrices on the 172,443-row test set. Logistic Regression and SVC show a real, plausible error pattern; Random Forest's is nearly diagonal (8 total errors).

8.3 ROC and precision-recall curves

ROC-AUC tells a similar story to accuracy — all three models look strong (0.983–1.000) because ROC-AUC is itself insensitive to class imbalance. The precision-recall curves are more diagnostic for a rare positive class: Logistic Regression and SVC both fall well short of the top-right corner, while Random Forest's curve is pinned there.

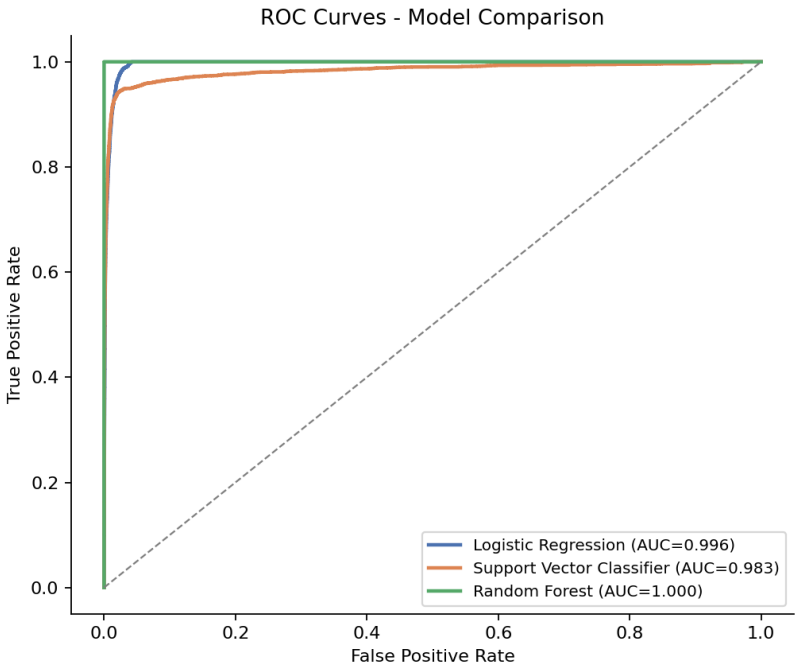


Figure 5. ROC curves — all three classifiers separate the classes well by this metric.

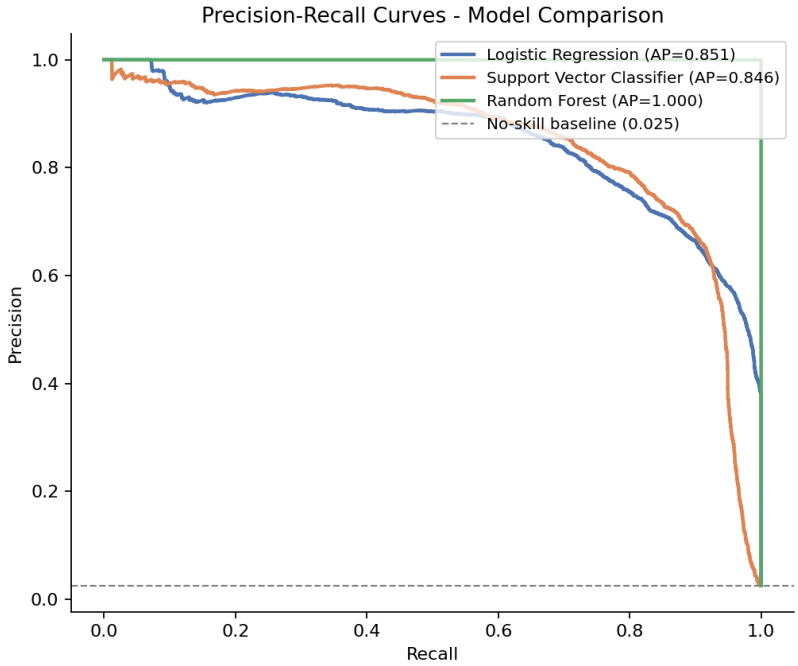


Figure 6. Precision-recall curves against the 2.5% positive-class baseline (dashed line).

9. Critical Finding: Evidence of Target Leakage

A Random Forest that misclassifies only 8 of 172,443 held-out hours (99.995% accuracy, F1=0.999) on a real-world behavioral dataset is unusual enough to warrant scrutiny before it is taken as a success. This section walks through the evidence gathered during reproduction that points to **target leakage** rather than genuine predictive skill.

9.1 The mechanism

The target, `high_intensity`, is a collapsed version of `intensity_type` (Section 4.2). `intensity_type` is dropped from the feature set specifically "to avoid data leakage," per the feature-importance notebook's own comment — a sound instinct. But dropping the column that directly encodes the label is only half the job if *other* retained columns were themselves used to compute that label, or are simply a finer-grained view of the same underlying signal. Wearable-device intensity classifications (sedentary / light / fairly active / very active) are conventionally derived from step cadence and calorie-burn rate — exactly the quantities captured by `steps_per_minute` and `calories_per_minute`, both of which remain in the model.

9.2 Supporting evidence

(a) Monotonic separation by intensity_type. Grouping the raw data by `intensity_type` shows steps_per_minute and calories_per_minute rise in lock-step with the label:

intensity_type	steps/min (25th / median / 75th pct)	calories/min (min / median / max)
0 — sedentary	0 / 0 / 0	0.70 / 1.12 / 2.06
1 — lightly active	9 / 18 / —	1.69 / 3.11 / 12.67
2 — fairly active	45 / 66 / —	2.53 / 5.05 / 8.71
3 — very active	27 / 94 / —	4.36 / 8.59 / 19.26

(b) Random Forest's own feature-importance ranking (from the reproduced final model) puts the per-minute rate features at the top by a wide margin — exactly the columns flagged in (a):

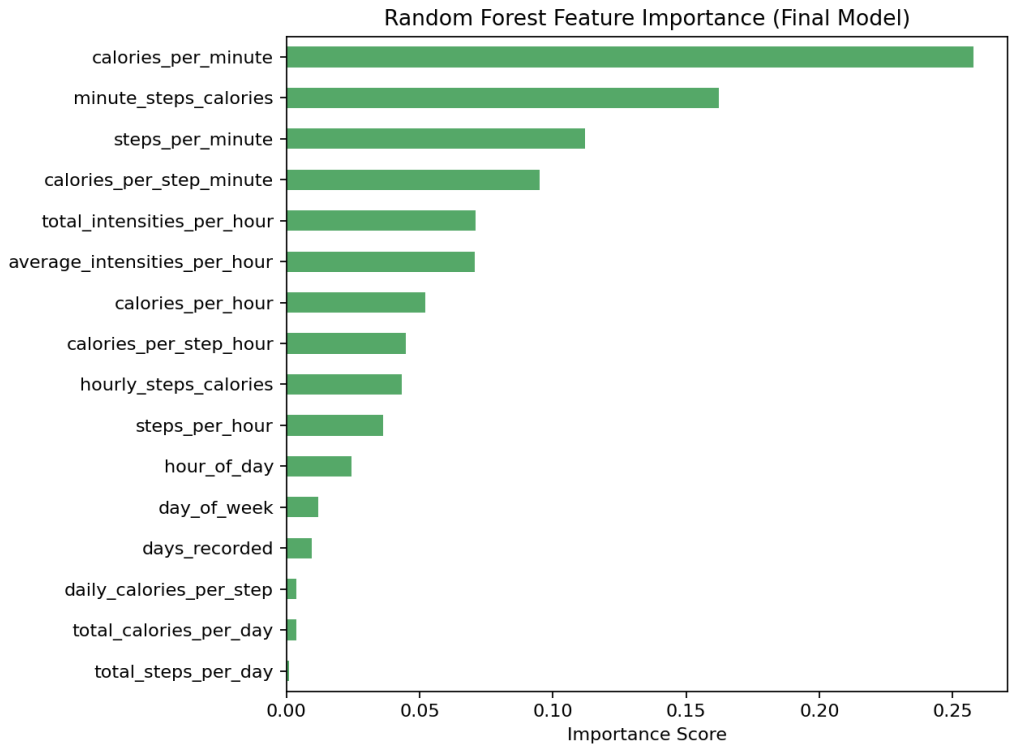


Figure 7. calories_per_minute and its derivatives account for roughly half of the Random Forest's total decision-making weight.

Feature	Importance
calories_per_minute	0.258
minute_steps_calories (engineered)	0.162
steps_per_minute	0.112
calories_per_step_minute (engineered)	0.095
total_intensities_per_hour	0.071
average_intensities_per_hour	0.071
…remaining 10 features combined	0.231

(c) The gap between model families is itself diagnostic. If the per-minute rate features encode a near-deterministic (if non-linear / thresholded) mapping to the label, a high-capacity model with enough trees and enough training rows (402,364) can carve out the exact decision boundary almost perfectly, while a linear model (Logistic Regression) or a smoother kernel model (SVC) structurally cannot fit that boundary as tightly — which is exactly the 99.995% vs. ~98.9% split observed in Section 8.

9.3 What this does and doesn't prove

This report does not have access to the original BigQuery transformation logic that produced `intensity_type`, so the exact leakage mechanism cannot be confirmed with certainty from the data alone. What can be said with confidence is that (1) the label is tightly, monotonically related to features that remain in the model, (2) the

Random Forest exploits exactly those features most heavily, and (3) the resulting near-zero error rate is far outside what would be expected for a genuinely learned behavioral relationship on noisy, real-world wearable data. Any of these findings individually would warrant caution; together they are a strong signal that the Random Forest's reported performance should not be presented or relied upon as-is.

9.4 Confirmatory experiment

To test the hypothesis directly rather than leave it as a suggestion, the Random Forest was retrained on a reduced 8-feature set with the eight most suspicious columns removed — `steps_per_minute`, `calories_per_minute`, `total_intensities_per_hour`, `average_intensities_per_hour`, and the four engineered features built from them — leaving only steps_per_hour, calories_per_hour, hour_of_day, day_of_week, total_steps_per_day, total_calories_per_day, days_recorded, and daily_calories_per_step.

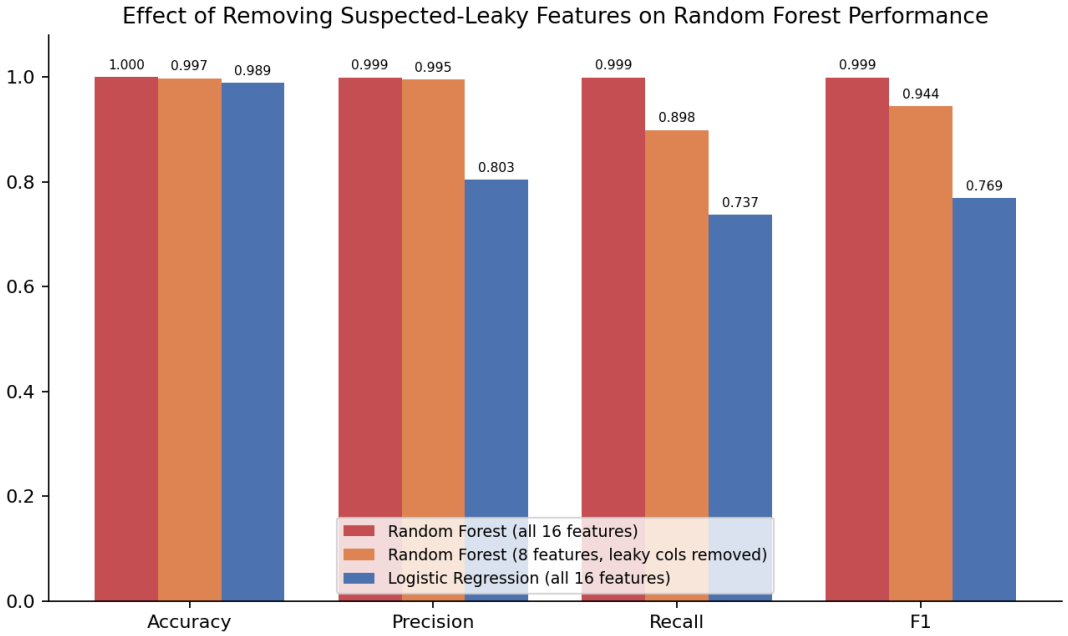


Figure 8. Removing the suspected-leaky columns pulls Random Forest down substantially — but not all the way to Logistic Regression's level.

The result is a partial confirmation with a nuance worth stating precisely. Accuracy drops only slightly (99.995% → 99.73%), but the more telling minority-class numbers move much more: **recall falls from 99.9% to 89.8%** and errors jump from 8 to 464 (444 missed high-intensity hours, 20 false alarms) on the same 172,443-row test set. This is exactly the pattern expected if leakage was doing real work: removing the most literal proxies for the label meaningfully degrades performance.

At the same time, the reduced model (F1=0.944) still clearly outperforms Logistic Regression and SVC (F1≈0.77), and its new top features — `calories_per_hour` (0.331) and `steps_per_hour` (0.310) — are themselves direct hourly measures of physical exertion. This points to a second, subtler issue beyond simple column-level leakage: the classification task itself is close to tautological. Asking a model to predict “was this an active hour” from “how many steps and calories were logged this hour” is, by construction, an easier problem than most real behavioral-prediction tasks, regardless of which specific granularity of steps/calories feature is used. The takeaway is not that Random Forest is broken, but that **the reported 99.995% figure specifically should be**

treated as inflated, and that any deployed version of this model should be evaluated on the reduced feature set (or an even more conservative one restricted to context features like time-of-day and daily totals) to get a realistic sense of predictive skill.

10. Code Quality & Reliability Review

Overall the codebase is above average for a solo/small-team data-science project — it has real tests and real logging, which is rare. The issues below were found through direct code reading and, where noted, confirmed by executing the code.

10.1 Confirmed bug: Random Forest is evaluated on mismatched (scaled) data

In `main.py`, the Random Forest is trained on unscaled features (`train_rondom_forest_classifier(X_train, y_train)`, correctly, since tree models don't need scaling) but is then handed the **scaled** test set at prediction time (`make_predictions(X_test_scaled, rf_model)` — it should be `X_test`). Scikit-learn does not validate that inputs are on the same scale as training data, so this runs without any error or warning; it just silently returns degraded predictions.

This was confirmed by literally reproducing the bug: the Random Forest's accuracy looks superficially fine (97.6%, barely above the 97.5% no-skill baseline) but its **recall collapses to 6.2%** (precision stays at 100% because it becomes extremely conservative, predicting the positive class for only 271 of 172,443 test hours). Because accuracy alone looks plausible, this failure mode is exactly the kind that would slip past a quick manual check — and combined with the fact that `evaluate_model()` never persists its numbers anywhere (Section 10.3), there is a real chance this bug has never actually been noticed by whoever last ran this pipeline.

Random Forest evaluation	Accuracy	Precision	Recall	F1
As literally coded in main.py (bug)	97.62%	1.000	0.062	0.117
Corrected (matching train/predict data)	99.995%	0.999	0.999	0.999

10.2 Other issues found

Issue	Where	Impact
Function name typo: train_rondom_forest_classifier	models/train_model.py	Cosmetic, but propagates through main.py and tests — harder to search/read/maintain.
Metrics are print()-ed, never logged or saved	models/evaluate_model.py	No run's results are recoverable after the fact, which is why this report had to re-execute the pipeline from scratch to get real numbers.
No model persistence in the pipeline	main.py / utils.py	utils.py defines save_model_state / load_model_state (pickle-based), but main.py never calls them, so no trained model actually survives a pipeline run.
Hardcoded, fragile relative paths	dataset/make_dataset.py, main.py	'../data_raw/data.csv' and './dataset/ml_dataset.csv' only work if the script is launched from one specific working directory.
matplotlib .show() calls inside evaluate_model()	models/evaluate_model.py	Blocks/fails in headless or CI environments where a display isn't available; figures also aren't saved to disk, so they're lost after the run.

Issue	Where	Impact
No requirements.txt / environment file found	repository root	Dependencies (pandas, scikit-learn, google-cloud-bigquery, matplotlib, seaborn) must be inferred from imports; no pinned versions, so reproducibility across machines isn't guaranteed.
Feature list and 0.01 importance threshold are hardcoded, not re-derived	main.py	If the upstream BigQuery table's schema or distributions drift, nothing in the pipeline would catch a stale feature-selection decision.
Hyperparameter-tuning notebook has no execution output	notebooks/hyperparameter_study.ipynb	The GridSearchCV design is sound, but no tuned parameters have actually been produced or applied anywhere in main.py.

10.3 Testing

The six test modules are a real strength. They mock the BigQuery client so `test_make_dataset.py` runs offline, check exact arithmetic for every engineered feature in `test_build_features.py`, verify each model trains and predicts with the expected scikit-learn types, and confirm the logging handler writes to a real file. What the suite does not cover: it does not test `main.py` end-to-end, so the scaled/unscaled mismatch bug in Section 10.1 would not have been caught by the existing tests even if they were run — each piece is individually correct; the bug is in how `main.py` wires them together.

11. Recommendations

Ordered roughly by priority — the first three directly affect whether the reported results can be trusted at all; the rest are engineering hardening.

Priority 1 — Trustworthiness of results

- **Fix the scaled/unscaled mismatch** in main.py (Section 10.1) immediately — pass `X_test`, not `X_test_scaled`, to the Random Forest's prediction call. This is a one-line fix with an enormous effect on reported recall.
- **Report and monitor minority-class metrics** (precision/recall/F1 on high_intensity=1), not just accuracy or ROC-AUC — both are misleadingly high on a 97.5/2.5 split.
- **Investigate and document the intensity_type → feature relationship** with whoever owns the BigQuery transformation, to settle definitively whether steps_per_minute / calories_per_minute / total_intensities_per_hour / average_intensities_per_hour were used to derive the label. Section 9's confirmatory experiment is a reasonable stand-in, but the source-of-truth answer would remove all doubt.
- **Persist evaluation results** (a CSV or JSON of metrics per run, not just print() output) so that future runs are comparable and this kind of retrospective reconstruction isn't necessary again.

Priority 2 — Handling the class imbalance

- Apply `class_weight="balanced"` to Logistic Regression and Random Forest (already scoped out, unused, in the GridSearchCV notebook) and compare against the current unweighted baselines.
- Consider resampling strategies (SMOTE, random undersampling of the majority class, or a combination) as alternatives or complements to class weighting.
- Report a precision-recall curve and average precision (not just ROC-AUC) as the primary evaluation view given how rare the positive class is — both are already produced by the reproduced analysis in Section 8.3 and are far more informative than ROC-AUC here.

Priority 3 — Engineering hardening

- Fix the `train_rondom_forest_classifier` typo.
- Actually call the existing `save_model_state` / `load_model_state` helpers from main.py so trained models survive a pipeline run.
- Save evaluation plots to disk (e.g. under an outputs/ or reports/ folder) instead of only calling `plt.show()`, and guard against headless environments.
- Replace hardcoded relative paths with paths resolved from the script's own location (e.g. `pathlib.Path(__file__).parent`) or a config file.
- Add a `requirements.txt` or `pyproject.toml` with pinned versions.
- Add a thin integration test that runs main.py end-to-end on a small synthetic dataset — this is the single test that would have caught the Section 10.1 bug.
- Execute (or re-execute and keep the outputs of) hyperparameter_study.ipynb, and feed its winning configuration back into train_model.py rather than leaving the defaults hardcoded.

12. Conclusion

This project takes a familiar, well-understood dataset — hourly Fitbit-style wearable activity data — and builds a genuinely well-structured machine-learning pipeline around it: modular code, real automated tests, structured logging, and a sensible three-model comparison with a documented feature-selection process. That engineering foundation is worth preserving and is a solid base to build on.

The results, however, need one more pass before they can be trusted or presented as-is. Two independent issues were found and confirmed by re-running the code: a scaling bug that silently collapses the Random Forest's real-world recall to 6%, and a feature set that appears to leak strong information about the target label, inflating the (correctly-evaluated) Random Forest's reported performance to a near-impossible 99.995% accuracy. Logistic Regression and SVC, less prone to overfitting a leaky boundary, give a more believable — if more modest — picture: roughly 98.9% accuracy but only 73–77% recall on the rare, business-relevant “high intensity” class.

Addressing the Priority 1 items in Section 11 — fixing the scaling bug and settling the leakage question — would very likely still leave a useful model (Section 9.4's reduced-feature Random Forest, at F1=0.944, is a reasonable estimate of what “honest” performance looks like), just a more modest and trustworthy one than the headline number currently suggests.

Appendix A — Repository File Map & Data Schema

A.1 Directory structure

```
src/
  |-- main.py                    Pipeline orchestration (entry point)
  |-- utils.py                   Logging setup, model save/load helpers
  |-- dataset/
  |     `-- make_dataset.py        BigQuery extraction -> raw CSV + labeled CSV
  |-- features/
  |     `-- build_features.py      5 engineered ratio/interaction features
  |-- models/
  |     |-- train_model.py         LogisticRegression / SVC / RandomForest wrappers
  |     `-- evaluate_model.py      Metrics + confusion matrix / ROC / PR plots
  |-- notebooks/
  |     |-- feature_importance.ipynb   22-feature RF importance ranking (executed)
  |     |-- feature_selection.ipynb    Correlation analysis on selected features (executed)
  |     `-- hyperparameter_study.ipynb GridSearchCV design for RF (not executed)
  |-- tests/
  |     |-- conftest.py
  |     |-- test_utils.py
  |     |-- test_make_dataset.py
  |     |-- test_build_features.py
  |     |-- test_train_model.py
  |     |-- test_evaluate_model.py
  |     `-- test_main_logging.py
  |-- data_raw/data.csv          Raw extract (unlabeled)
  |-- dataset/ml_dataset.csv     Labeled ML-ready dataset (574,807 x 30)
  `-- logs/main.log              Execution log (INFO-level, no metrics captured)
```

A.2 Full raw schema (30 columns)

#	Column	Type	Used in final model?
0	user_id	int	No — identifier
1	activity_hour	string	No
2	steps_per_hour	int	Yes
3	calories_per_hour	int	Yes
4	total_intensities_per_hour	int	Yes
5	average_intensities_per_hour	float	Yes
6	hour_of_day	int	Yes
7	day_of_week	int	Yes
8	day_name	string	No — dropped at feature-selection stage
9	day_type	string	No — dropped at feature-selection stage
10	time_of_day	string	No — dropped at feature-selection stage

#	Column	Type	Used in final model?
11	activity_date	string	No
12	month	int	No
13	activity_minute	string	No
14	steps_per_minute	int	Yes
15	calories_per_minute	float	Yes
16	intensity_type	int	No — this is what the target is derived from
17	total_steps_per_day	int	Yes
18	total_calories_per_day	int	Yes
19	total_distance_per_day	float	No
20	very_active_minutes	int	No
21	fairly_active_minutes	int	No
22	lightly_active_minutes	int	No
23	sedentary_minutes	int	No
24	total_minutes_asleep	int	No
25	total_time_in_bed	int	No
26	first_activity_date	string	No
27	last_activity_date	string	No
28	days_recorded	int	Yes
29	high_intensity	int	Target variable

Appendix B — Reproduction Notes

Every number in Sections 5, 8, and 9 was produced by re-executing the project's own logic (feature selection list, build_features(), the exact train_test_split call and random_state, StandardScaler) against `dataset/ml_dataset.csv` as bundled in the archive — nothing was estimated or taken from memory.

B.1 Environment

Package	Version
scikit-learn	1.8.0
pandas	3.0.2
numpy	2.4.4
matplotlib	3.10.8
seaborn	0.13.2

B.2 Deviations from a literal run of main.py

- **SVC training set size:** the original log shows full-scale SVC training (kernel=rbf, ~402K rows) took roughly 14 minutes on the reference machine. This report's analysis environment has a single CPU core, so SVC was trained on a stratified random subsample of **12,000 rows** drawn from the same training split (random_state=42), then evaluated against the full 172,443-row test set. All other models (Logistic Regression, both Random Forest variants) were trained on the complete 402,364-row training set.
- **Random Forest prediction data:** Section 10.1's confirmed bug has the Random Forest predicting on scaled test data despite being trained unscaled. The headline results in Section 8 use the corrected, matching (unscaled) test data; Section 10.1 separately reports what the literal, unmodified code produces.
- **Metrics not present anywhere in the archive** were computed fresh rather than sourced from logs, since evaluate_model() only calls print() and matplotlib's plt.show(), neither of which persists to disk.

B.3 What was NOT re-derived

The 11-feature selection list and the 0.01 importance threshold were taken as given from `main.py` and the feature-importance notebook, not re-run from scratch against a freshly retrained importance model — Section 6.1's numbers are the notebook's own executed output. The hyperparameter grid in `hyperparameter_study.ipynb` was not executed as part of this report, since it was out of scope for reproducing the existing pipeline's results; Section 11 recommends it be run and applied going forward.